



Lecture 9: Component Props

INTERVIEW QUESTION

[> CORE](#)

How do you pass data from a parent component into a child?

You've built a `Button` component, but every instance looks identical. How do you give each one its own label?

Components allow you to encapsulate logic and styling, but a component is only truly useful if it is reusable. If every instance of a component is perfectly identical, its usefulness is severely limited.

The Problem with Hardcoded Components

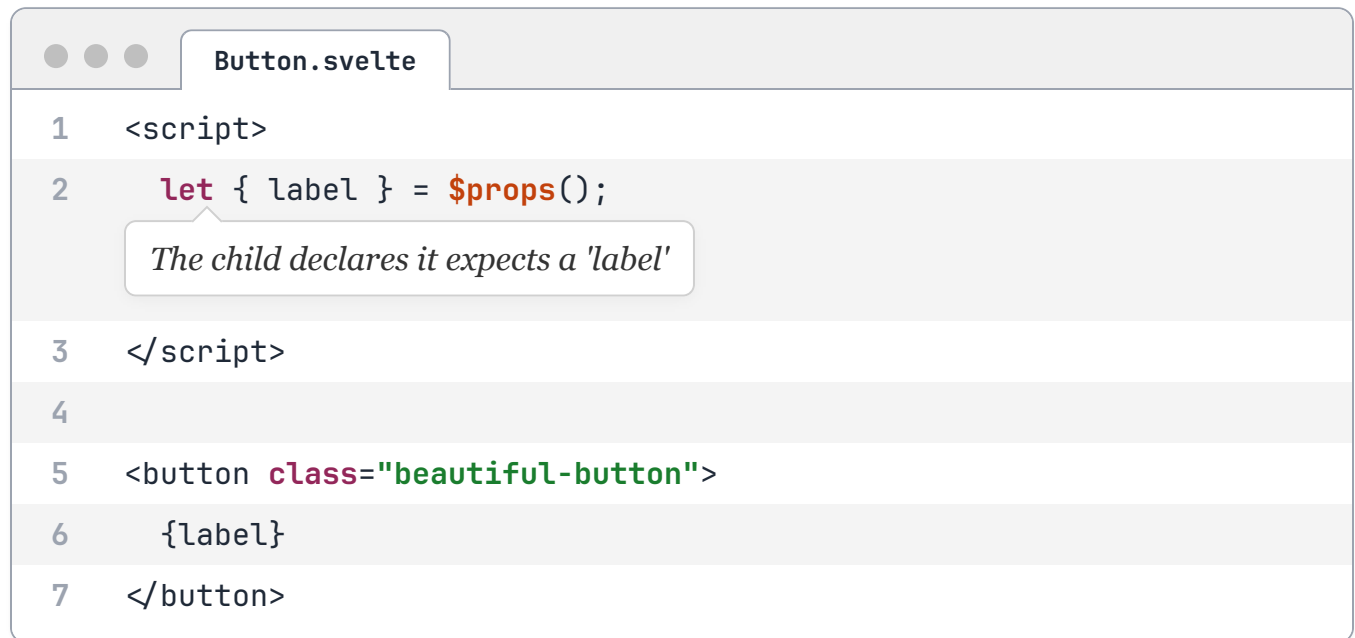
Imagine you extract a beautiful, perfectly styled `<Button>` into its own file. It has the right drop shadow, the right hover state, and the right typography. But inside `Button.svelte`, you hardcoded the text `Click Me`.

When you import that button into your dashboard, you need one button that says "Save", one that says "Delete", and one that says "Submit". Because the text is hardcoded inside the child, every single button says "Click Me". To fix this without props, you would have to duplicate the entire file into `SaveButton.svelte`, `DeleteButton.svelte`, and `SubmitButton.svelte`. This completely destroys the point of reusable components.

The `$props` Rune

To make components flexible, Svelte 5 provides the `$props` rune. Props (short for properties) act as the inputs to your component. They allow a parent component to pass data down into a child component.

Inside the child component, you declare that you expect to receive data by destructuring the `$props()` rune.



```
1 <script>
2   let { label } = $props();
3 </script>
4
5 <button class="beautiful-button">
6   {label}
7 </button>
```

The child declares it expects a 'label'

Passing Props from the Parent

Now that the child component is ready to receive a `label`, the parent component can pass that data in. You pass props to a Svelte component exactly the same way you pass attributes to a standard HTML element.



```
1  <script>
2    import Button from './Button.svelte';
3  </script>
4
5  <div class="actions">
6    <Button label="Save" />
7    <Button label="Delete" />
8  </div>
```

The parent passes a static string

Props are not limited to static strings. You can pass reactive `$state` variables, numbers, objects, or even arrays by using curly braces. Because props are tied directly into Svelte's reactivity system, if the parent's state changes, the child automatically updates.

● ● ● Dashboard.svelte

```
1 <script>
2   import Button from './Button.svelte';
3   let dynamicText = $state("Submit");
4 </script>
5
6 <Button label={dynamicText} />
```

The parent passes a reactive variable

KEY TAKEAWAY

Props flow strictly downwards: from parent to child. The parent owns the data, and the child merely receives it to render the UI.

Summary

To make components reusable, they need to accept dynamic inputs called props. In Svelte 5, the child component declares its expected inputs using `let { propName } = $props()` . The parent component passes data into those props using standard HTML attribute syntax. This downward flow of data allows you to build a single, highly flexible component (like a generic `<Button>`) and reuse it in dozens of different contexts just by changing the data you feed it.

	HTML ATTRIBUTES	COMPONENT PROPS
Recipient	Native DOM elements (<code><input></code>)	Svelte components (<code><Button></code>)
Declaration	Built into the browser	Declared via <code>let { ... } = \$props()</code>
Data Types	Mostly strings and booleans	Any JavaScript type (Objects, Arrays)